

Desenvolvimento para Frontend

Introdução à TypeScript

Universidade Federal do Rio Grande do Norte

O que é TypeScript?

- TypeScript é um superset do JavaScript que adiciona tipagem estática ao JavaScript.
 - Qualquer código JavaScript válido também é um código TypeScript válido
- O TypeScript foi desenvolvido pela Microsoft e é de código aberto.
 - Código-fonte disponibilizado publicamente para que qualquer pessoa possa ver, modificar e distribuir. (<https://github.com/microsoft/TypeScript>)
 - Licença Apache 2.0
- TypeScript é projetado para desenvolvimento de grandes aplicações e compila para JavaScript.

Vantagens do TypeScript

- **Tipagem Estática:** Permite a detecção de erros durante o desenvolvimento, em vez de em tempo de execução, aumentando a confiabilidade do código.
- **Desenvolvimento Escalável:** Facilita a manutenção de grandes bases de código, promovendo uma arquitetura de código mais limpa e estruturada.
- **Ferramentas de IDE Melhoradas:** Proporciona uma melhor experiência de desenvolvimento com autocompletar, navegação de código e refatoração mais eficientes.
- **Suporte para ES6 e Além:** Permite usar as mais recentes funcionalidades do JavaScript sem se preocupar com compatibilidade de navegador, pois o TypeScript é compilado para JavaScript compatível com versões anteriores.

Como Começar

- O TypeScript pode ser **instalado** globalmente usando npm (Node Package Manager) com o comando `npm install -g typescript`.
 - Para compilar um arquivo TypeScript (.ts) em JavaScript (.js), use o comando **`npx tsc arquivo.ts`** para executar use-se o comando **`node arquivo.js`**.
 - <https://www.typescriptlang.org/play/>
- A configuração de um projeto TypeScript é feita criando um arquivo **`tsconfig.json`** que especifica as opções do compilador e os arquivos a serem incluídos com o comando **`npx tsc --init`**

Tipagem Básica – Tipos primitivos

- TypeScript suporta os seguintes tipos primitivos:
 - boolean, string, number, null, undefined, any, never, symbol, void e bigint.

```
let isDone: boolean = true;
let decimal: number = 6;
let hex: number = 0xf00d;
let binary: number = 0b1010;
let octal: number = 0o744;
let color: string = "blue";
let fullName: string = `Robert Marks`;
let age: number = 37;
let sentence: string = `Meu nome é ${fullName}. Minha idade é ${age + 1}.`;
```

Tipagem Básica – Arrays e Tuplas

- Arrays podem ser tipados usando `tipo[]` ou `Array<tipo>`.
- Arrays permitem uma coleção de dados do mesmo tipo.

```
let list1: number[] = [1, 2, 3];  
let list2: Array<number> = [1, 2, 3];
```

- **Tuplas** permitem definir um **array com um número fixo de elementos de tipos específicos**. Cada posição no array pode ter um tipo diferente.

```
// define our tuple  
let ourTuple: [number, boolean, string];  
// initialize correctly  
ourTuple = [5, false, 'Coding God was here'];
```

Tipagem Básica – Enums

- Enums permitem definir um conjunto de valores nomeados, facilitando o trabalho com conjuntos de constantes relacionadas.

```
enum Cor { Vermelho, Verde, Azul }  
let c: Cor = Cor.Verde;
```

```
enum Color { Red = 10, Green = 1, Blue = 3 }  
let colorName: string = Color[10];  
console.log(colorName); // 'Red'
```

Tipagem Básica – Any

- O tipo any é uma “saída” para quando não sabemos de antemão quais tipos uma variável pode ter. Permite a migração gradual de código JavaScript existente para TypeScript.

```
let notSure: any = 4;  
notSure = “nome e sobrenome”;  
notSure = false; // também é permitido
```

- Flexibilidade no início da migração de grandes projetos, ao custo de perder a verificação de tipos

Interfaces

- Interfaces são contratos dentro do código que definem a estrutura que os objetos devem seguir. São usadas para tipar a forma exata de objetos.
- Benefícios:
 - **Verificação de Tipos:** Ajuda a verificar se os objetos possuem a estrutura esperada.
 - **Documentação:** Serve como documentação para descrever como os objetos devem ser estruturados e usados.
 - **Reutilização:** Interfaces podem ser reutilizadas em diferentes partes do código, promovendo consistência.

Tipagem Básica – Interfaces

```
interface RequestMessage {  
    text: string;  
}  
  
function printMessage(requestMessageObj: RequestMessage) {  
    console.log(requestMessageObj.text);  
}  
  
let example = {size: 10, text: "Hello World"};  
  
printMessage(example);
```

- A função `printMessage` espera um **objeto que adere à interface** `RequestMessage`, garantindo que o objeto passado tenha uma propriedade **text** do tipo **string**.

Interfaces - propriedades Opcionais e Readonly

- Propriedades Opcionais são denotadas com ? e permitem a flexibilidade de omitir certas propriedades ao criar objetos.

```
interface SquareConfig {  
  color?: string;  
  width?: number;  
}  
  
function createSquare(config: SquareConfig): {color: string; area: number} {  
  let newSquare = {color: "white", area: 100};  
  if (config.color) {  
    newSquare.color = config.color;  
  }  
  if (config.width) {  
    newSquare.area = config.width * config.width;  
  }  
  return newSquare;  
}  
  
let mySquare = createSquare({color: "black"});
```

Interfaces - propriedades Opcionais e Readonly

- Propriedades readonly só podem ser modificadas na criação do objeto, tornando-as imutáveis posteriormente.

```
interface SomeType {  
  readonly prop: string;  
}  
  
function doSomething(obj: SomeType) {  
  // We can read from 'obj.prop'.  
  console.log(`prop has the value '${obj.prop}'.`);  
  
  // But we can't re-assign it.  
  obj.prop = "hello";  
  //Cannot assign to 'prop' because it is a read-only property.  
}
```

Atividade 1

- Crie uma interface para representar um objeto de usuário com as propriedades: id (número), nome (string), e email (string). Em seguida, escreva uma função que aceita um objeto que implemente essa interface e imprime as informações do usuário.

Atividade 1

- Crie uma interface para representar um objeto de **usuário** com as propriedades: id (número), nome (string), e email (string). Em seguida, escreva uma função que aceita um objeto que implemente essa interface e imprime as informações do usuário.

```
interface Usuario {  
  id: number;  
  nome: string;  
  email: string;  
}  
  
function imprimirUsuario(usuario: Usuario): void {  
  console.log(`ID: ${usuario.id}`);  
  console.log(`Nome: ${usuario.nome}`);  
  console.log(`Email: ${usuario.email}`);  
}  
  
const user: Usuario = { id: 1, nome: 'João', email: 'joao@example.com' };  
imprimirUsuario(user);
```

Atividade 2

- Defina um enum chamado DiaDaSemana com os dias da semana. Crie uma função que recebe um valor do enum e imprime o nome do dia correspondente.

Atividade 2

- Defina um enum chamado DiaDaSemana com os dias da semana. Crie uma função que recebe um valor do enum e imprime o nome do dia correspondente.

```
enum DiaDaSemana {  
    Domingo,  
    Segunda,  
    Terca,  
    Quarta,  
    Quinta,  
    Sexta,  
    Sabado  
}  
  
function imprimirDia(dia: DiaDaSemana): void {  
    console.log(DiaDaSemana[dia]);  
}  
  
imprimirDia(DiaDaSemana.Quarta); // Saída: Quarta
```


Atividade 3

- Crie uma tupla para representar um ponto 3D (x, y, z) e uma função que aceita essa tupla e imprime as coordenadas.

Atividade 3

- Crie uma tupla para representar um ponto 3D (x, y, z) e uma função que aceita essa tupla e imprime as coordenadas.

```
type Ponto3D = [number, number, number];

function imprimirPonto(ponto: Ponto3D): void {
  console.log(`x: ${ponto[0]}, y: ${ponto[1]}, z: ${ponto[2]}`);
}

const ponto: Ponto3D = [10, 20, 30];
imprimirPonto(ponto); // Saída: x: 10, y: 20, z: 30
```

Referências

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/>
- <https://www.typescriptlang.org/docs/>
- <https://www.geeksforgeeks.org/>